

# Web Programming

Rodrigo Martins Pagliares

Last update: 2021/06/08

# Web Programming

Rodrigo Martins Pagliares

# Web programming foundations

# The Document Object Model

# Introducing the Document Object Model

Lecture: Introducing the Document Object Model  
Module: The Document Object Model

The **DOM** (Document Object Model) is a **hierarchical, structural representation** of the **elements** in a **web document** and their **relationships**.

# The Document Object Model

- When we're talking about the **content** that's being **rendered** in the **browser**, we are **talking about** the **DOM**.
  - ✓ The Document Object Model.
- The **DOM** is the **browser's model** for the **document displayed**.

DOM

# The Document Object Model

- The **DOM** is the **way** you **access** **elements** on the **page** with the **other** **languages**.
- If the **DOM** isn't **coded** **correctly**, it can **break** the **entire** **website**.

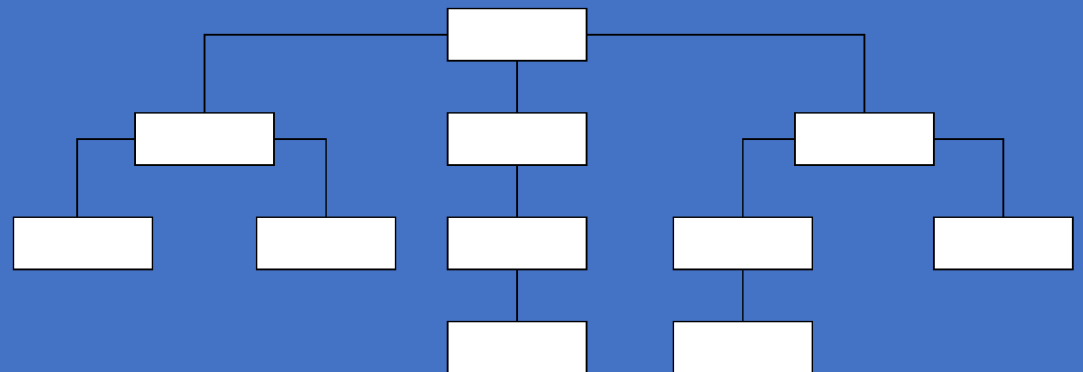


Source: <https://bit.ly/3icq2Qh>

**how does the browser “see”  
a web document?**

# The Document Object Model

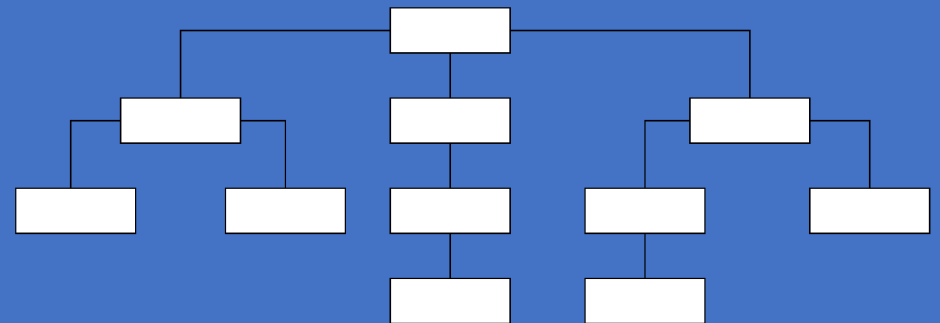
- Any time a browser is asked to display an HTML document, it creates a DOM.
- The DOM can be visualized as a DOM tree:
  - ✓ Each element is a leaf or node (a.k.a. element, object).
  - ✓ Connections are branches.



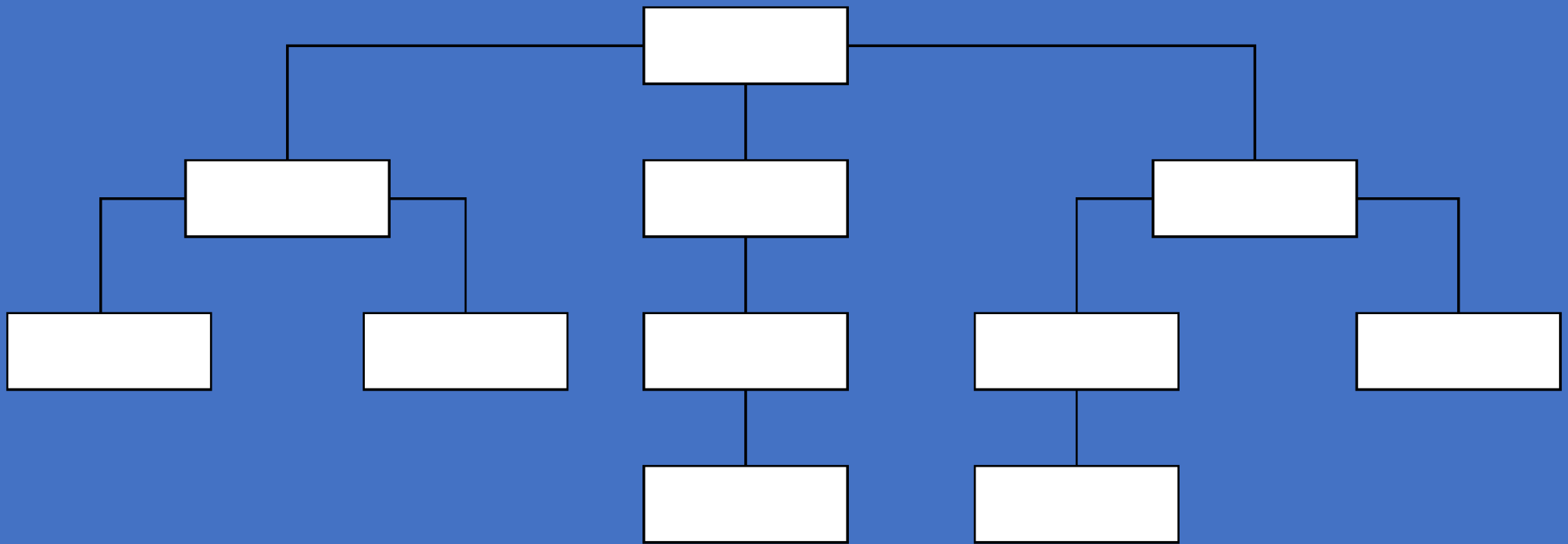
# The Document Object Model

- Every piece of **HTML** content is **wrapped** in a **beginning** and **end tags**, creating an **HTML element**.
- Each of these elements is a **DOM object (node)**.
  - ✓ The **browser** handles each of them as an **independent object**.

Each **HTML** element  
is a **node**  
in the **DOM** tree.



# The DOM Tree



Legend:

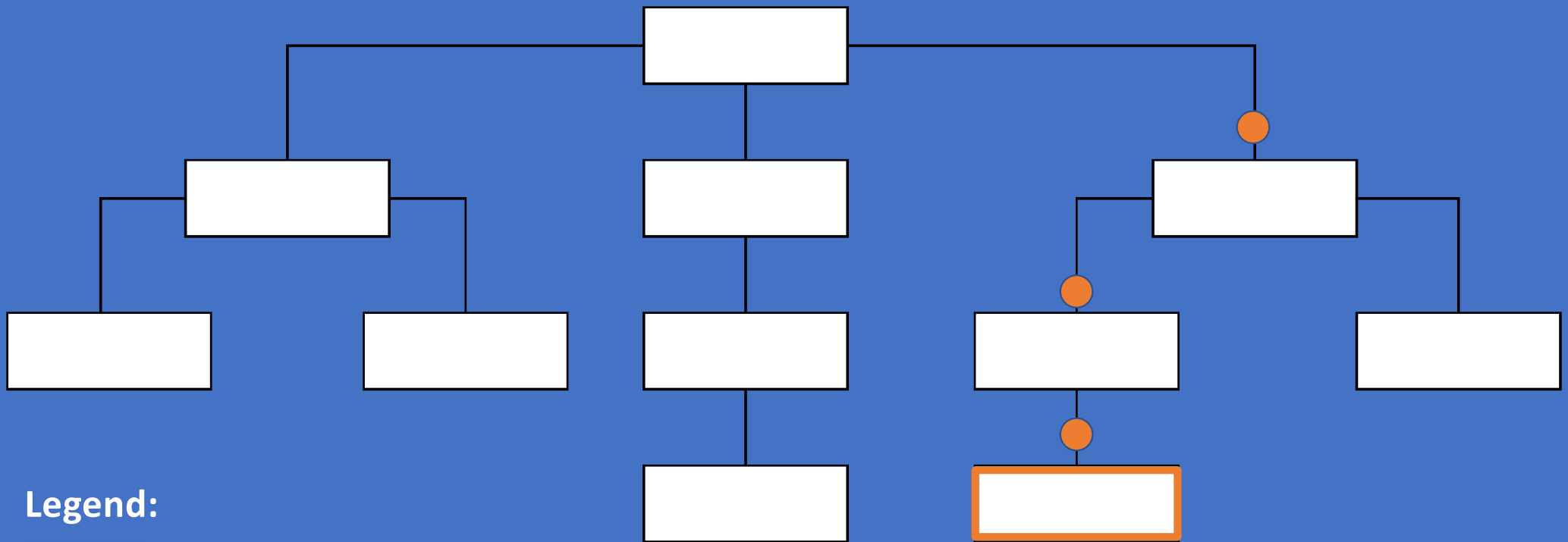


Nodes



Branches

# Walking, traversing, or navigating the DOM tree



## Legend:

**Nodes**

☐ Element found

## Branches

# The DOM tree

- When you add new elements to an **HTML document**, you are **adding a node** to the **DOM tree**.



Source: <https://bit.ly/3pfRnme>

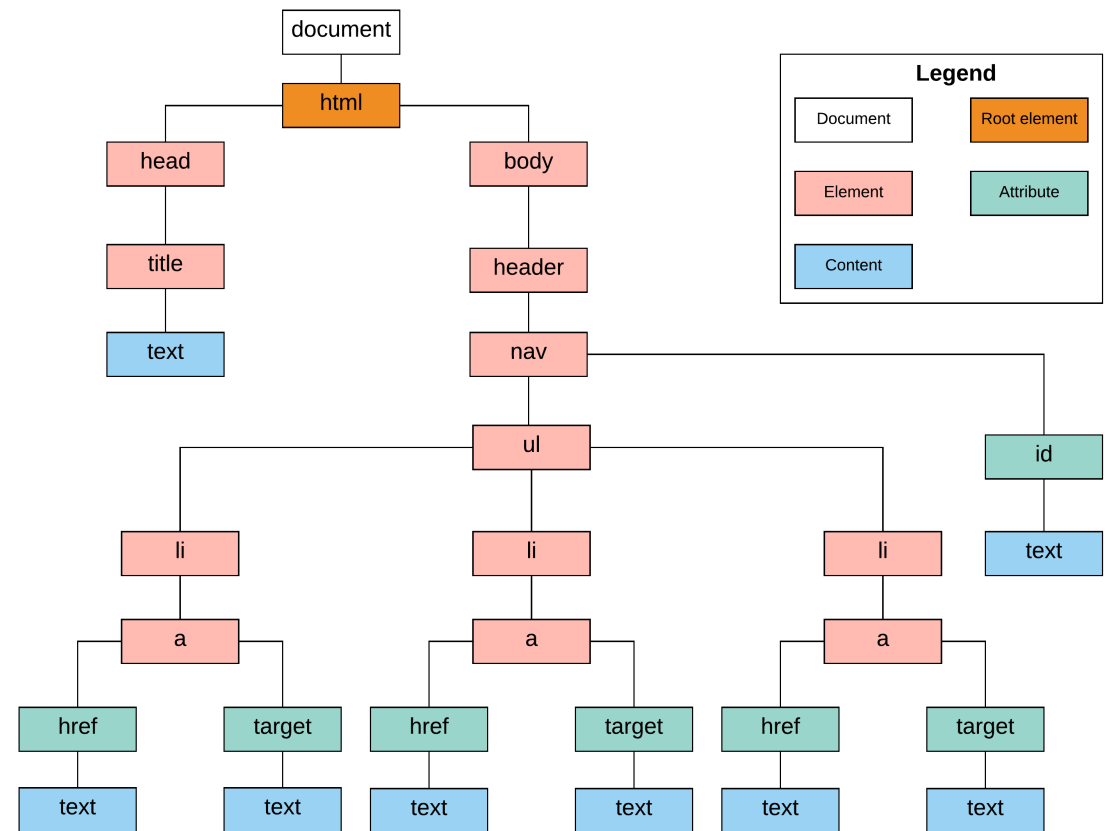
# The Document Object Model

- The **browser** creates the **DOM** based on the **semantic markup** in the **HTML** document.
- **CSS** rules and **JavaScript** apply **presentational** and **behaviors** to **elements** and **content** in the **document** by **targeting** these **nodes** in the **DOM**.

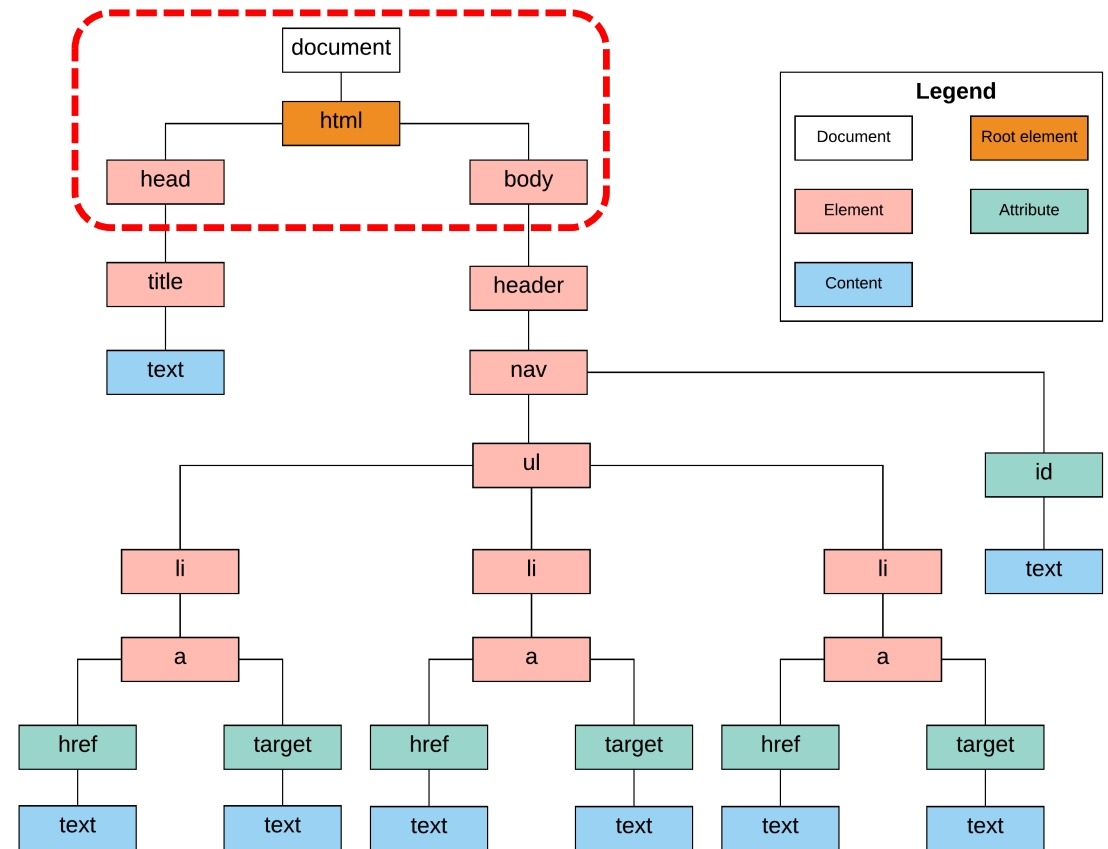


Source: <https://bit.ly/3icq2Qh>

Sometimes it is useful to visualize or draw out the DOM tree or part of it to understand the relationships and inheritance between different elements.

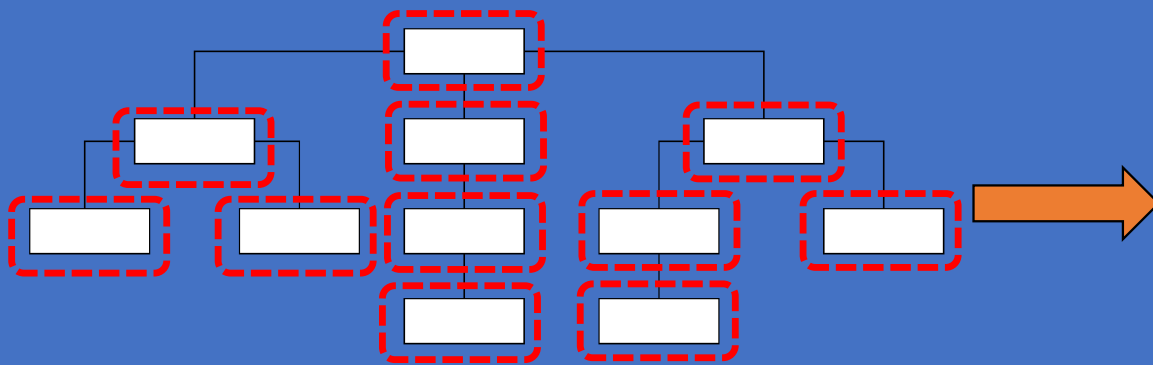


In **HTML**, the **document** itself is the parent node of the **HTML** element. **HEAD** and **BODY** are child nodes of the **HTML** element.



# DOM

- Eventually, it becomes natural to break down everything you see in a browser as a logical grouping of boxes.
- Boxes represent nodes on a DOM tree.
- Nodes represent tags in an HTML document.



Source: <https://bit.ly/3pn3sWU>

# HTML & the DOM

Lecture: HTML & the DOM

Module: The Document Object Model

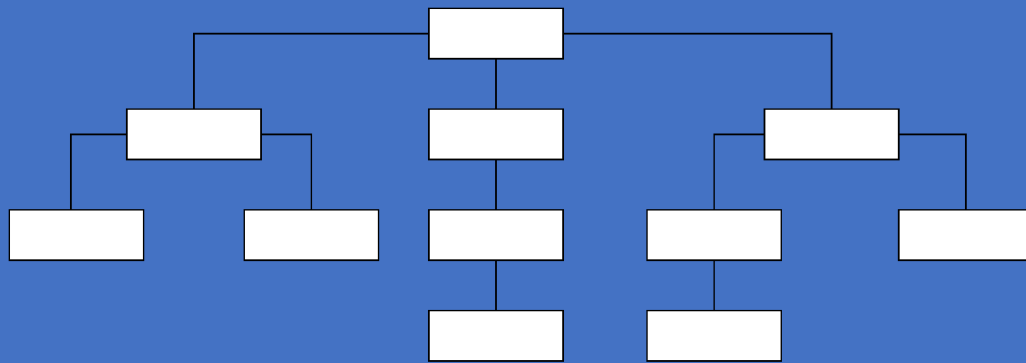
# Markup language and the DOM

- Markup generates the DOM.
- The **DOM** describes the **structure** of the **page**
- When you add **markup tags** to a **document**, you're **building structure** that **defines** the **role** of your **content**.
  - ✓ This **structure** is **important** to how **CSS** and **JavaScript** work.

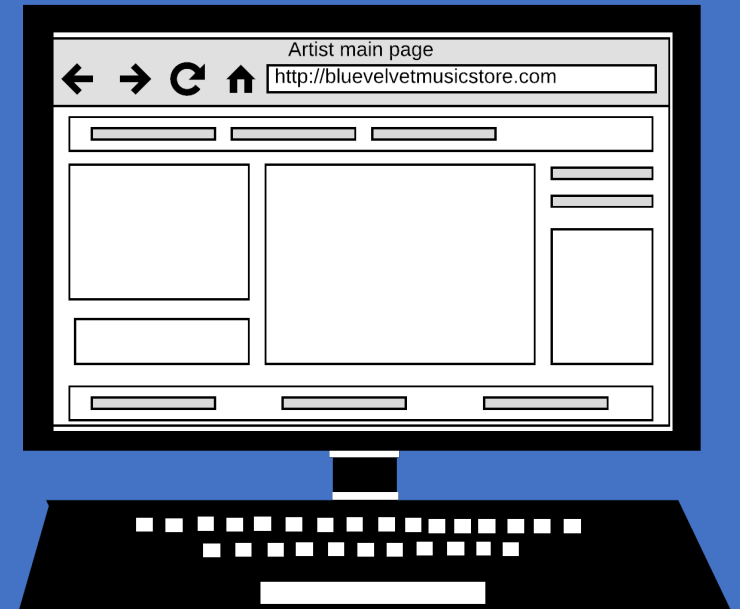
# HTML and the DOM

- A link in a menu is a **node nested** inside a `<li>` **node**, nested inside a `<ul>` **node**, nested inside a `<nav>` **node**, nested inside a `<header>` **node**, nested inside the `<body>` **node**, nested inside the `<html>` **node**.

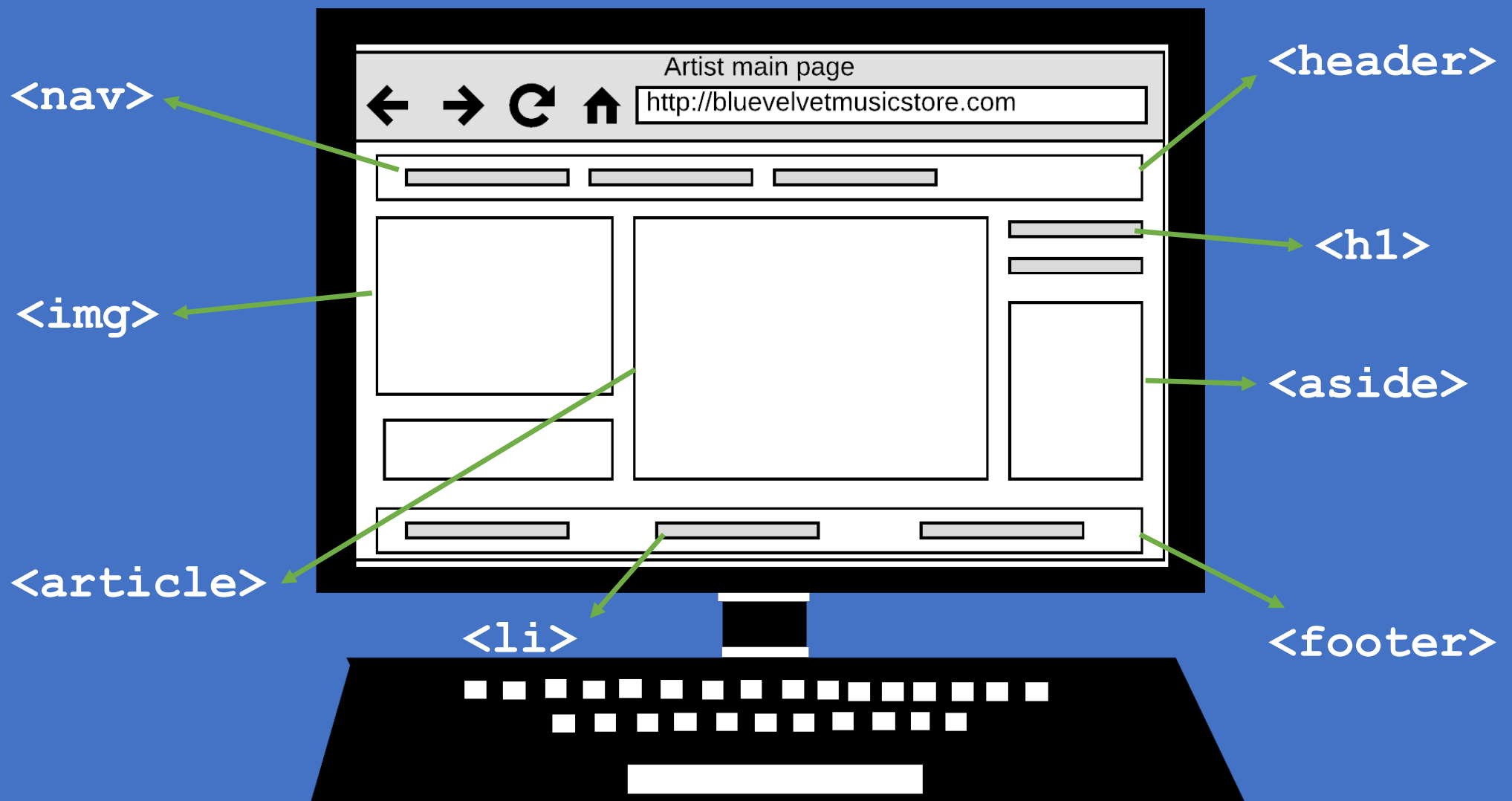
```
<> html-dom-relationship.html •
<> html-dom-relationship.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>HTML and the DOM</title>
5  </head>
6  <body>
7  |   <header>
8  |   |   <nav id="main_menu">
9  |   |   |   <ul>
10 |   |   |   |   <li><a href="https://mzl.la/2ufSlqm" target="_blank"> HTML</a></li>
11 |   |   |   |   <li><a href="https://mzl.la/2uSVblo" target="_blank">CSS </a></li>
12 |   |   |   |   <li><a href="https://mzl.la/3b9LkZ9" target="_blank">JavaScript</a></li>
13 |   |   |   </ul>
14 |   |   </nav>
15 |   </header>
16 </body>
</html>
```



**Document Object Model  
(DOM)**



**HTML rendered  
in the viewport**



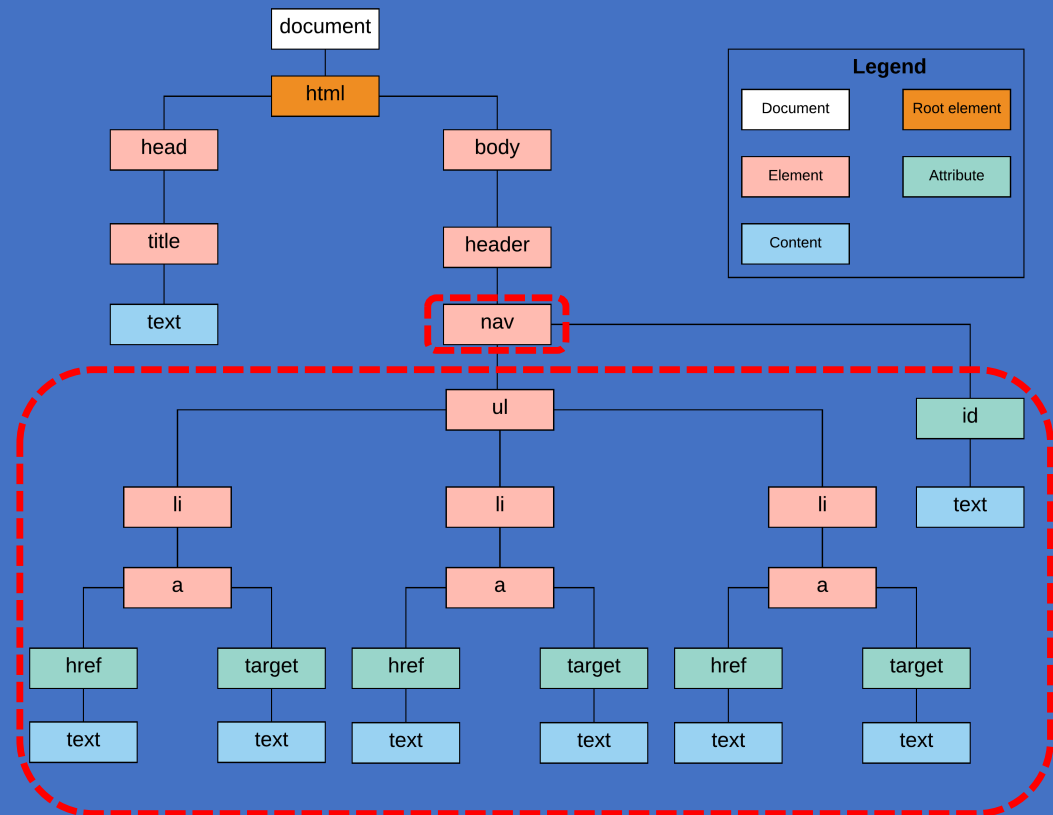
# DOM

- The **HTML structure** in this example is **human-readable** and clearly shows the **relationship** between the **different elements** through **nesting**.

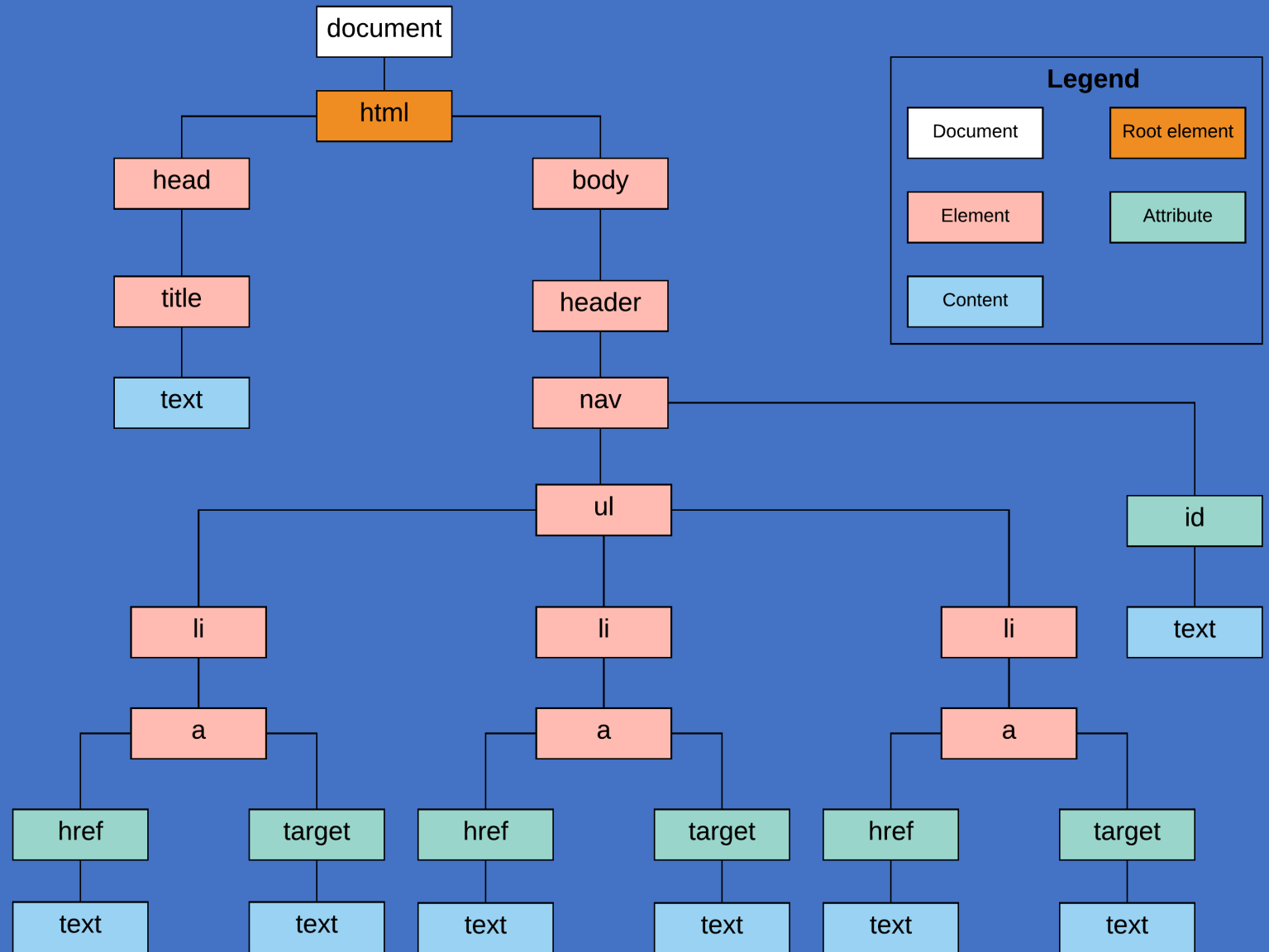
```
<> html-dom-relationship.html •
<> html-dom-relationship.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>HTML and the DOM</title>
5  </head>
6  <body>
7  |   <header>
8  |   |   <nav id="main_menu">
9  |   |   |   <ul>
10 |   |   |   |   <li><a href="https://mzl.la/2ufSlqm" target="_blank"> HTML</a></li>
11 |   |   |   |   <li><a href="https://mzl.la/2uSVblo" target="_blank">CSS </a></li>
12 |   |   |   |   <li><a href="https://mzl.la/3b9LkZ9" target="_blank">JavaScript</a></li>
13 |   |   |   </ul>
14 |   |   </nav>
15 |   </header>
16 </body>
</html>
```

# DOM

- Based on the HTML nesting, we can draw out the DOM tree for the `<nav>` node
- Web designers often make use of these drawings to visualize the relationships before the coding.
  - ✓ Either in their heads, in a paper, or in a board.



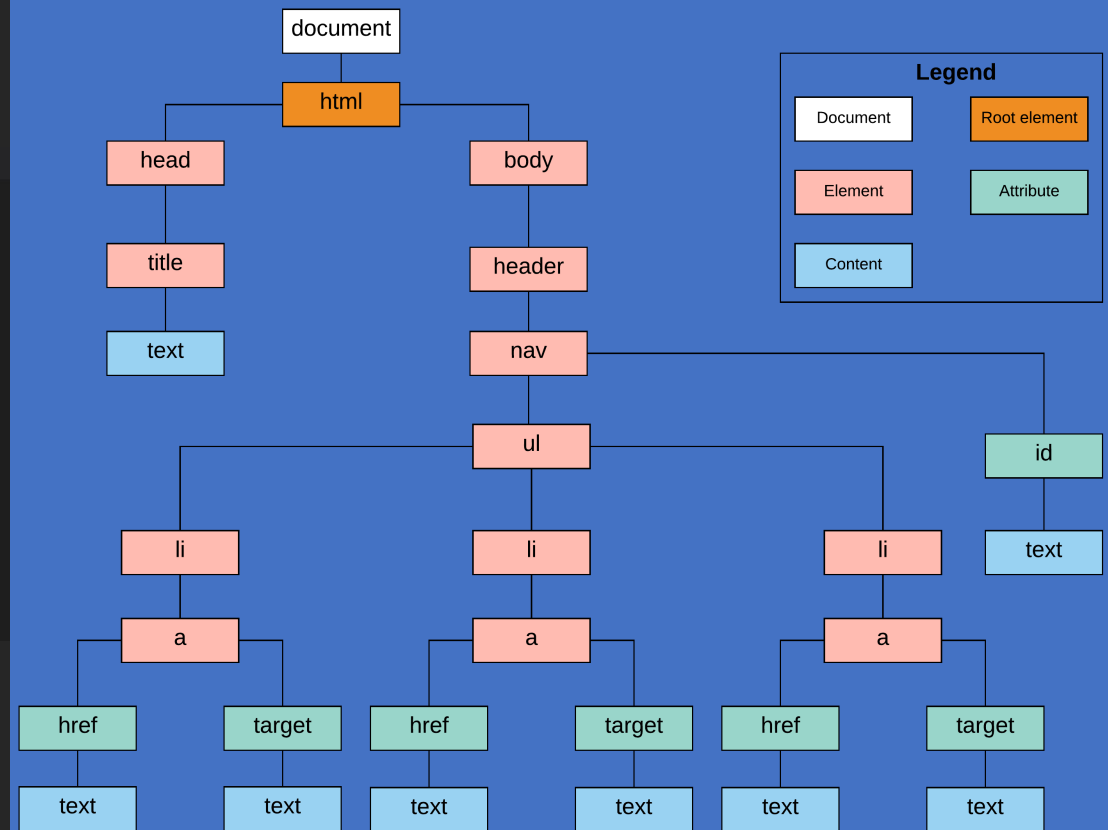
# DOM Tree

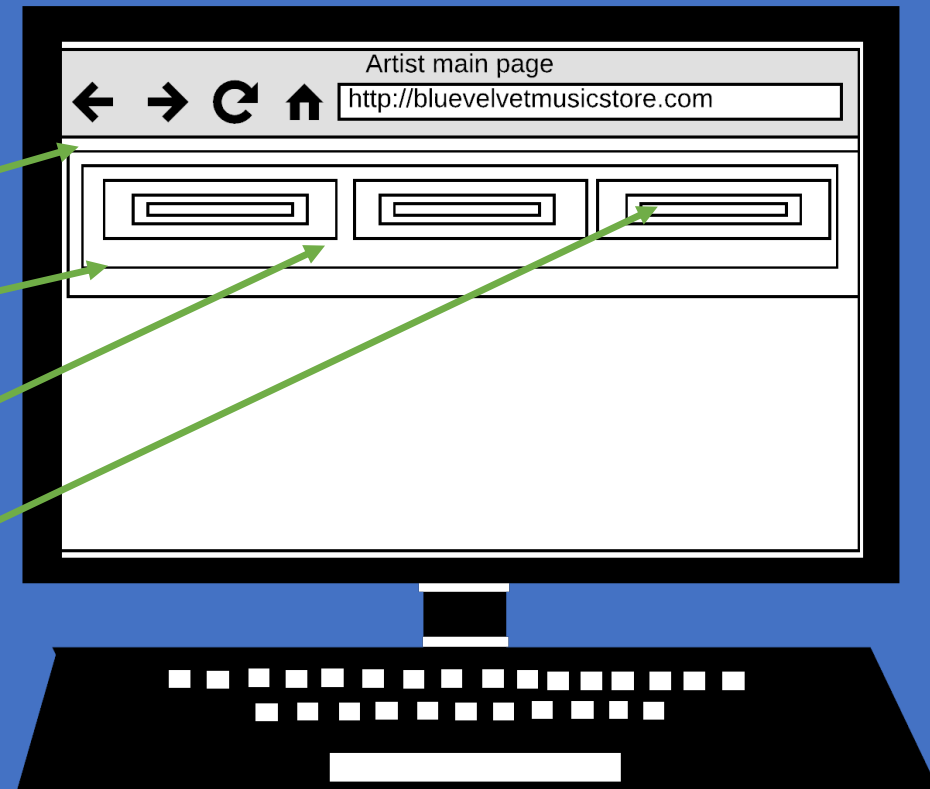
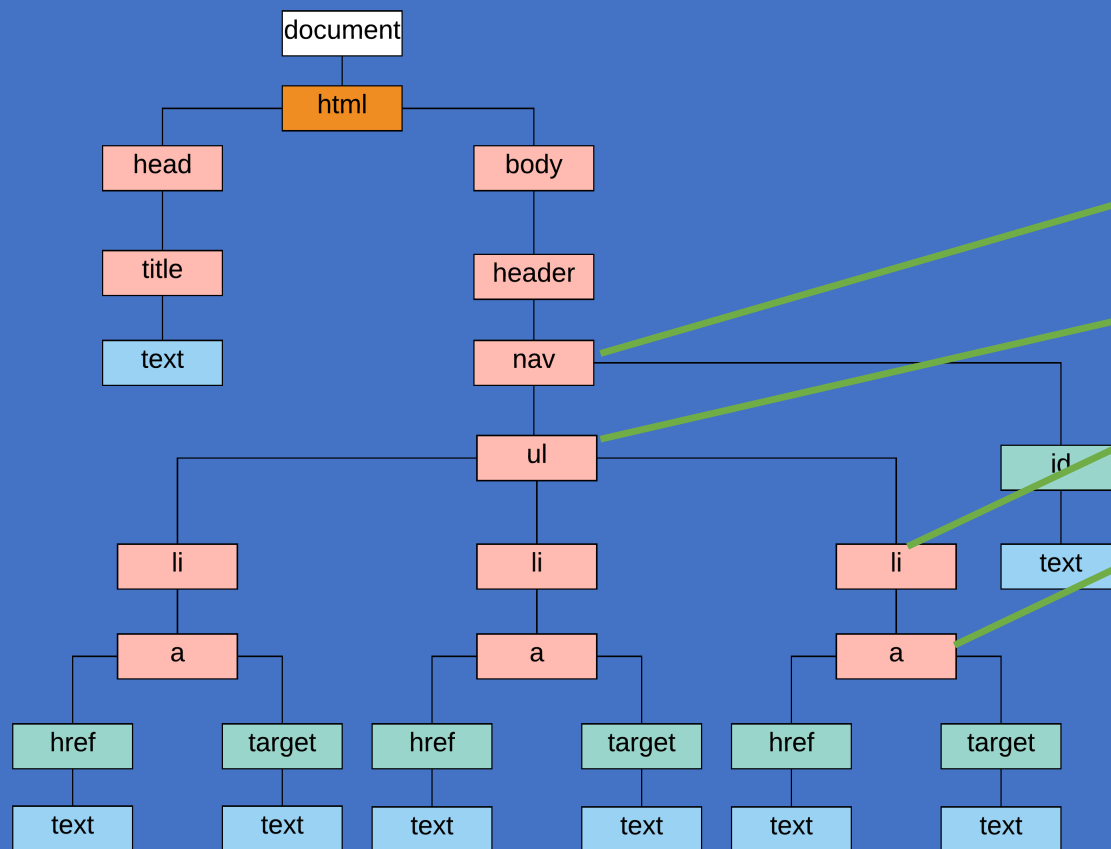


# HTML DOCUMENT

```
html-dom-relationship.html •
html-dom-relationship.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>HTML and the DOM</title>
5  </head>
6  <body>
7    <header>
8      <nav id="main_menu">
9        <ul>
10       <li><a href="https://mzl.la/2ufSlqm" target="_blank"> HTML</a></li>
11       <li><a href="https://mzl.la/2uSVblo" target="_blank">CSS </a></li>
12       <li><a href="https://mzl.la/3b9LkZ9" target="_blank">JavaScript</a></li>
13     </ul>
14   </nav>
15 </header>
16 </body>
</html>
```

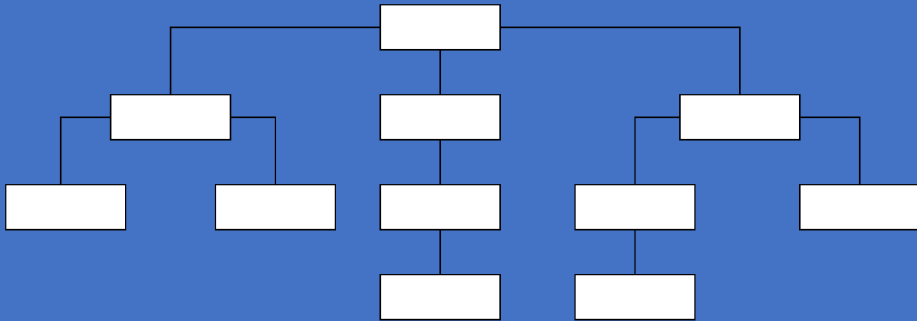
# DOM TREE





It's all about boxes

Remembering that **every element is a box**, and that **every box is a node on the tree connected to its parents, siblings, and descendants via branches** is key to understanding how the browser sees the Web and how you can give the browser instructions to do what you want.



Source: <https://bit.ly/3pn3sWU>

# JavaScript & the DOM

Lecture: JavaScript & the DOM

Module: The Document Object Model

**JavaScript runs small programs in the browser that interact with the DOM by targeting DOM nodes and reacting to browser events.**

# JavaScript and the DOM

- JavaScript loads into the browser to add interactive behaviors and make changes to DOM nodes.

```
<> apply-javascript-external.html > ...
1  <!DOCTYPE html>
2  <html lang="en-US">
3    <head>
4      <meta charset="utf-8">
5      <title>Apply JavaScript example</title>
6      <script src="script.js" defer></script>
7    </head>
8    <body>
9      <button>Click me</button>
10   </body>
11 </html>
```

## HTML

```
JS script.js > createParagraph
1  function createParagraph() {
2    let para = document.createElement('p');
3    para.textContent = 'You clicked the button!';
4    document.body.appendChild(para);
5  }
6
7  const buttons = document.querySelectorAll('button');
8
9  for(let i = 0; i < buttons.length ; i++) {
10    buttons[i].addEventListener('click', createParagraph);
11  }
12
```

## JavaScript

# CSS & the DOM

Lecture: CSS & the DOM

Module: The Document Object Model

# CSS and inheritance

- if an **element** is given a **style property** and it has a **descendant**, so a **sub element**, that **descendant inherits the style property**.
  - ✓ Unless another rule overrides it.
- **Example:**
  - ✓ The `<body>` **element** is the **topmost element** in the **visual part** of the **DOM tree**.
  - ✓ if we set the **font-family** and **color** properties for the **body element**, those properties apply to all content within that document.

# CSS inheritance and the DOM

```
<> css-example.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8">
5      <link rel="stylesheet" type="text/css" href="css/styles.css">
6      <title>Document</title>
7    </head>
8
9    <body>
10     <p> This a simple example to demonstrate style inheritance n CSS</p>
11   </body>
12 </html>
```

HTML

```
css > # styles.css > ...
1  body {
2    font-family: sans-serif;
3    color: ■ red;
4  }
5
6  p {
7    font-family: verdana;
8  }
9
```

CSS

# Targeting DOM nodes with CSS

## CSS selectors

used to **target DOM nodes**  
(**body** and **p** in this example)

```
css > # styles.css > ...  
1  body {  
2      font-family: sans-serif;  
3      color: red;  
4  }  
5  
6  p {  
7      font-family: verdana;  
8  }  
9
```

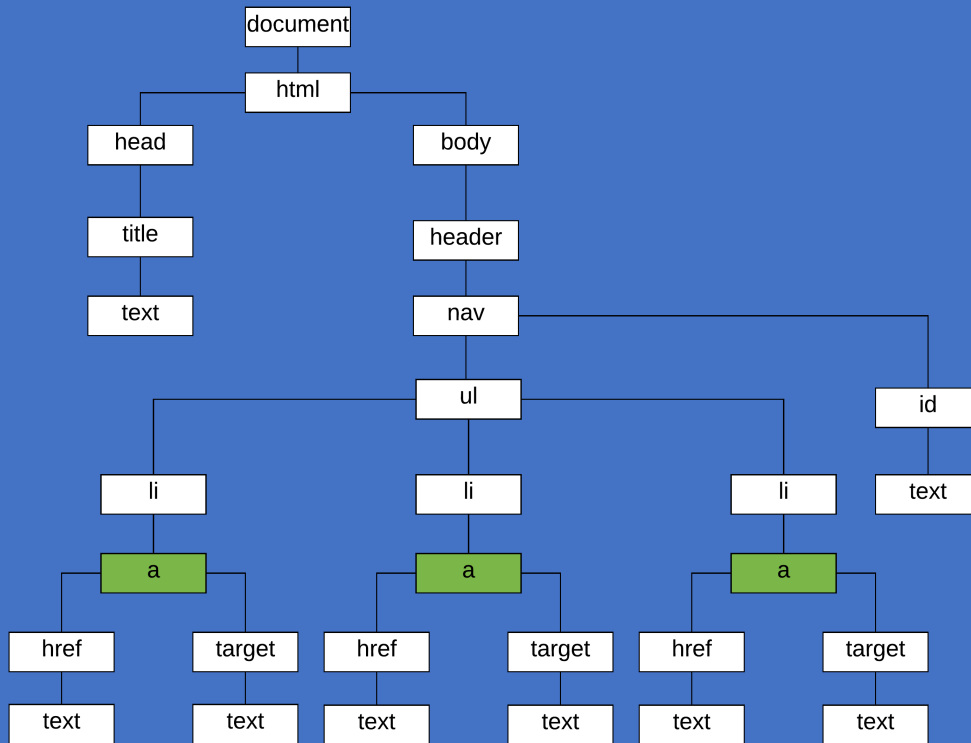
# CSS inheritance and override

```
<> css-example.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <link rel="stylesheet" type="text/css" href="css/styles.css">
6          <title>Document</title>
7      </head>
8
9      <body>
10         <p> This a simple example to demonstrate style inheritance n CSS</p>
11     </body>
12 </html>
```

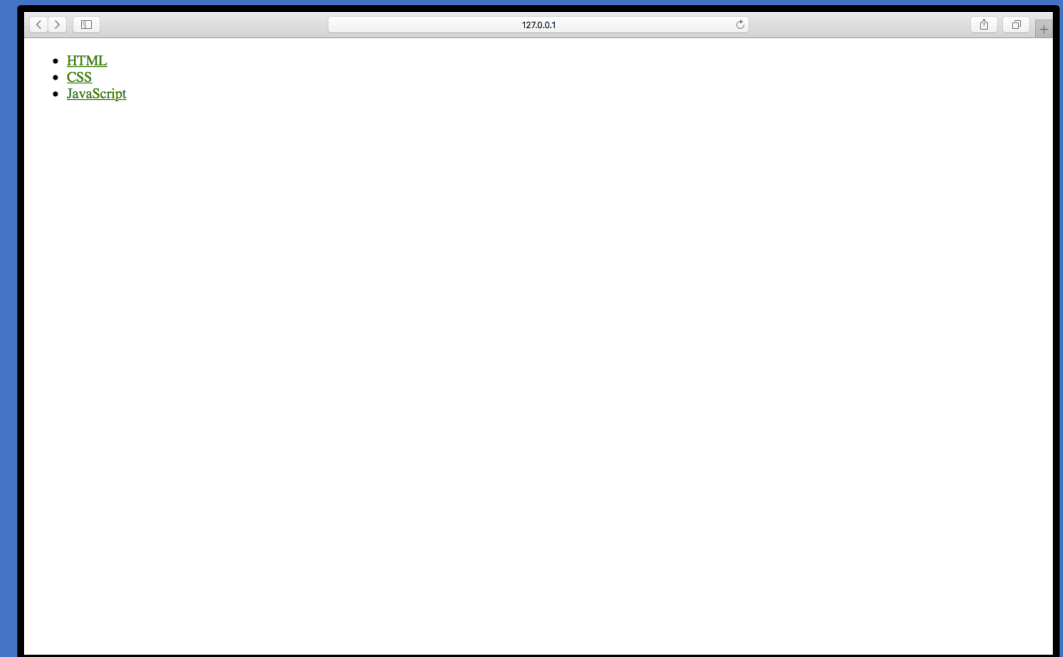
```
css > # styles.css > ...
1  body {
2      font-family: sans-serif;
3      color: ■ red;
4  }
5
6  p {
7      font-family: verdana;
8  }
9
```

Descendants of any element inherits its style properties. Here, all fonts become red. Paragraphs become red through inheritance and their font are overridden with Verdana font.

# Find every link in the page changing their font color to green.



```
# styles.css •
css > # styles.css > ...
1  a {
2      color: green;
3  }
4
```



# Cascade

Rules are applied in a **top-down** fashion. The rules **farthest down** override the previous ones.

```
# styles.css •
css > # styles.css > ...
1  ✓ a {
2    |   color: ■ green;
3    |   }
4
5  ✓ a {
6    |   color: ■ red
7    |   }
```



All links are red

# DOM events

Lecture: DOM events

Module: The Document Object Model

**“Events are actions or occurrences that happen in the system you are programming, which the system tells you about so you can respond to them in some way if desired.”**

Source: [https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building\\_blocks/Events](https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Events)

# Dom events and JavaScript

- At any time, when something happens in a browser, an event is triggered, and JavaScript can use these events to trigger further behaviors.
- Everything that happens in the browser is registered as an event:
  - ✓ Open the browser.
  - ✓ Enter a URL in the address bar and hit enter.
  - ✓ Click a link.
  - ✓ Move your mouse over the viewport.
  - ✓ ...

# Events

- **Whenever you interact with the browser, you fire an event and the browser reacts to that event with pre-configured responses.**
- **Using JavaScript, we can write our own scripts to respond to these events as well.**

# Event handling

- The **approach** for **binding JavaScript functionality** around **events** is called **event handling**.
- **Event handling:**
  1. **Identify** the **DOM node** the **interactive behavior** will be **attached to**.
    - For example, a HTML **button** element.
  2. **Identify** the **event** you **want to detect** and **bind** it to the **DOM node**.
    - A **click**, a **hover**, etc.
  3. Create a **function** that is **triggered** when the **event fires**.

**Recap in 5 steps**

# Recap in 5 steps

1. The **HTML markup** describes the document and **identifies** each **unique object** within it using **semantic tags**.
2. The **semantic tags** are turned in **nodes** in the **DOM tree**.
  - ✓The **nodes** can be seen as **boxes**.
3. **JavaScript** targets **DOM nodes** to **change** and **apply interactivity** to them.

## Recap in 5 steps

4. CSS rules target **DOM** nodes and **apply** presentational styling.

5. **Events** are **triggered** when the **user**, **browser** or **document** instance performs some sort of **behavior**.

- ✓ At this **step**, the **CSS**, **HTML** and **DOM** tree may get changed.

Questions?

Questions?